Containers, Extensions, and Morphisms: The Representation Theorem

Extracted from a machine-checked development

Abstract

We present the elementary theory of *containers* in the sense of Abbott, Altenkirch, and Ghani: a shape type equipped with a family of position types. Each container determines a polynomial endofunctor on types—its *extension*. Morphisms of containers induce natural transformations between extensions, and conversely every natural transformation between extensions arises uniquely from a container morphism. The proofs of all stated results have been machine-checked; we record only the mathematical content.

Contents

1 Introduction **1**

2 Containers and their extensions **2**

3 Morphisms of containers **3**

4 The representation theorem **4**

5 Discussion **5**

1 Introduction

A recurring pattern in data-type theory is that a polymorphic type constructor is determined by a collection of *shapes* together with, for each shape, a set of *positions* at which payload values may be stored. Lists are determined by a natural number (the length) and one position per index; binary trees by a shape tree and positions at the leaves or nodes; and so on.

Containers make this pattern precise. A container is simply a type of shapes and a dependent family of positions. Its *extension* is the associated polynomial functor: an element of the extension at a type X is a shape together with an X-labelling of the positions of that shape. Container morphisms—maps on shapes and *contravariant* maps on positions—correspond exactly to natural transformations between extensions. That correspondence is the content of the representation theorem (Theorem 4.8 below).

The material is elementary and self-contained. We work throughout in a single universe of types, writing **Type** for that universe. No special categorical machinery beyond the notions of functor and natural transformation is required. The statements below are those of a formal development in dependent type theory; proofs are indicated by the phrase “by lean,” meaning they have been checked by a proof assistant rather than written out in prose.

What this note claims. In ordinary mathematical language, the claim is: polynomial functors of the form

$$X \longmapsto \sum_{s \in S} X^{P(s)}$$

and the natural transformations between them, are completely classified by the combinatorial data (S, P) of shapes and positions, together with shape-and-position maps in the sense of Definition 3.1. There is nothing more to a natural transformation between two such functors than a map of shapes and a family of position pullbacks.

2 Containers and their extensions

Definition 2.1 (Container)**.** A **container** C consists of

1. a type $\mathrm{Shape}(C)$ of *shapes*, and
2. for each shape $s \in \mathrm{Shape}(C)$, a type $\mathrm{Pos}(C)(s)$ of *positions* of s.

We write $C = \big(\mathrm{Shape}(C),\, \mathrm{Pos}(C)\big)$, or more briefly $C = (S, P)$ when the names of the components are clear.

Example 2.2.

1. **Lists.** Let $\mathrm{Shape} = \mathbb{N}$ and $\mathrm{Pos}(n) = \{0, \dots, n-1\}$. The extension (defined below) at X is the type of finite lists of elements of X.
2. **Streams.** Let $\mathrm{Shape} = 1$ (a singleton) and $\mathrm{Pos}(*) = \mathbb{N}$. The extension at X is the type of infinite streams of elements of X.
3. **Binary trees with values at leaves.** Shapes are full binary tree skeletons; positions of a skeleton are its leaves.

Definition 2.3 (Extension)**.** Let $C = (S, P)$ be a container and let X be a type. The **extension** of C at X is the dependent sum

$$\mathrm{Ext}\, C\, X \;=\; \sum_{s\,:\,S} \big(P(s) \to X\big). \tag{1}$$

An element of $\mathrm{Ext}\, C\, X$ is a pair $\langle s,\, v\rangle$ consisting of a shape $s \in S$ and a function $v : P(s) \to X$ labelling the positions of s by elements of X.

Thus $\mathrm{Ext}\, C$ is the polynomial functor associated to the family P.

Definition 2.4 (Functorial action)**.** Let C be a container and let $f : X \to Y$ be a function. The **functorial action** of the extension on f is the map

$$\mathrm{Ext}\, C\, f \;:\; \mathrm{Ext}\, C\, X \;\longrightarrow\; \mathrm{Ext}\, C\, Y \tag{2}$$

defined by relabelling positions:

$$\mathrm{Ext}\, C\, f\, \langle s,\, v\rangle \;=\; \langle s,\; f \circ v\rangle. \tag{3}$$

The shape is left unchanged; only the labels are post-composed with f.

Proposition 2.5 (Identity law)**.** *For every container C and every type X,*

$$\mathrm{Ext}\, C\, \mathrm{id}_X \;=\; \mathrm{id}_{\mathrm{Ext}\, C\, X}. \tag{4}$$

Proof. By lean. □

Proposition 2.6 (Composition law)**.** *For every container C and functions $f : X \to Y$, $g : Y \to Z$,*

$$\operatorname{Ext} C\,(g \circ f) \;=\; \operatorname{Ext} C\, g \;\circ\; \operatorname{Ext} C\, f. \tag{5}$$

Proof. By lean. □

Taken together, Propositions 2.5 and 2.6 say that $\operatorname{Ext} C$ is an endofunctor of **Type**. The proofs are immediate from the definitions: both sides act by post-composition of the labelling function, and post-composition preserves identities and composition.

3 Morphisms of containers

A morphism of containers must induce a natural transformation of extensions. Because an element of an extension is a shape plus a labelling of positions, mapping from $\operatorname{Ext} C$ to $\operatorname{Ext} D$ requires sending C-shapes to D-shapes and *pulling* D-positions back to C-positions (so that a C-labelling can be reindexed to a D-labelling). The position component is therefore contravariant.

Definition 3.1 (Container morphism)**.** Let $C = (S_C, P_C)$ and $D = (S_D, P_D)$ be containers. A **morphism of containers** $\varphi : C \Rightarrow D$ consists of

1. a map on shapes $u : S_C \to S_D$, and
2. for each $s \in S_C$, a function
$$g_s \;:\; P_D\big(u(s)\big) \;\longrightarrow\; P_C(s)$$
(a *contravariant* map on positions).

We write $\varphi = (u, g)$ when we need the components explicitly.

Remark 3.2. The variance on positions is forced by the intended semantics. Given a labelling $v : P_C(s) \to X$ of a C-shape, and a D-position i of the image shape $u(s)$, one obtains a label by composing

$$i \;\longmapsto\; g_s(i) \;\longmapsto\; v\big(g_s(i)\big).$$

Thus positions of the target are pulled back to positions of the source.

Definition 3.3 (Induced map on extensions)**.** Let $\varphi = (u, g) : C \Rightarrow D$ be a container morphism and let X be a type. The **induced map**

$$\varphi^*_X \;:\; \operatorname{Ext} C\, X \;\longrightarrow\; \operatorname{Ext} D\, X \tag{6}$$

is defined by

$$\varphi^*_X\langle s,\, v\rangle \;=\; \langle u(s),\; v \circ g_s\rangle. \tag{7}$$

Proposition 3.4 (Naturality)**.** *For every container morphism $\varphi : C \Rightarrow D$ and every function $f : X \to Y$, the following square commutes:*

$$\operatorname{Ext} D\, f \;\circ\; \varphi^*_X \;=\; \varphi^*_Y \;\circ\; \operatorname{Ext} C\, f. \tag{8}$$

In other words, the family $\big(\varphi^*_X\big)_X$ is a natural transformation $\operatorname{Ext} C \Longrightarrow \operatorname{Ext} D$.

Proof. By lean. □

The naturality square says that it does not matter whether one first applies the container morphism and then relabels, or first relabels and then applies the morphism: both routes send $\langle s, v\rangle$ to $\langle u(s),\; f \circ v \circ g_s\rangle$.

4 The representation theorem

We now state the correspondence between container morphisms and natural transformations of extensions. First we package the latter as a definition.

Definition 4.1 (Natural transformation of extensions)**.** Let C and D be containers. A **natural transformation** $\eta : \operatorname{Ext} C \Longrightarrow \operatorname{Ext} D$ is a family of maps

$$\eta_X \;:\; \operatorname{Ext} C\, X \;\longrightarrow\; \operatorname{Ext} D\, X, \tag{9}$$

indexed by types X, such that for every $f : X \to Y$,

$$\operatorname{Ext} D\, f \;\circ\; \eta_X \;=\; \eta_Y \;\circ\; \operatorname{Ext} C\, f. \tag{10}$$

Definition 4.2 (From morphisms to natural transformations)**.** Every container morphism $\varphi : C \Rightarrow D$ determines a natural transformation

$$\varphi^* \;:\; \operatorname{Ext} C \Longrightarrow \operatorname{Ext} D \tag{11}$$

whose component at X is the map φ^*_X of Definition 3.3. Naturality holds by Proposition 3.4.

The converse direction recovers a container morphism by probing a natural transformation at a carefully chosen "generic" element. Fix a shape $s \in \operatorname{Shape}(C)$. Consider the type of positions $P_C(s)$ itself as the payload type, and form the element

$$\langle s,\, \mathrm{id}_{P_C(s)} \rangle \;\in\; \operatorname{Ext} C\, (P_C(s)). \tag{12}$$

This is the *generic element* at shape s: it labels each position by itself. Applying η to it yields a shape of D and a labelling of that shape's positions by C-positions—precisely the data of a container morphism.

Definition 4.3 (From natural transformations to morphisms)**.** Let $\eta : \operatorname{Ext} C \Longrightarrow \operatorname{Ext} D$ be a natural transformation. Define a container morphism $\varphi = (u, g) : C \Rightarrow D$ by

$$u(s) \;=\; \pi_1\Big(\eta_{P_C(s)} \langle s,\, \mathrm{id} \rangle\Big), \tag{13}$$

$$g_s \;=\; \pi_2\Big(\eta_{P_C(s)} \langle s,\, \mathrm{id} \rangle\Big), \tag{14}$$

where π_1 and π_2 are the projections of the dependent sum $\operatorname{Ext} D\, (P_C(s))$. We write $\varphi = \eta^\sharp$ for this morphism.

Theorem 4.4 (Morphism round-trip)**.** *Let $\varphi : C \Rightarrow D$ be a container morphism. Recovering a morphism from the induced natural transformation returns the original:*

$$(\varphi^*)^\sharp \;=\; \varphi. \tag{15}$$

Proof. By lean. □

Remark 4.5. This direction is purely computational. Evaluating φ^* on the generic element $\langle s, \mathrm{id} \rangle$ yields $\langle u(s),\, g_s \rangle$ by definition of φ^*, so reading off shape and position maps recovers (u, g) immediately. No appeal to naturality is required.

Theorem 4.6 (Natural transformation round-trip)**.** *Let $\eta : \operatorname{Ext} C \Longrightarrow \operatorname{Ext} D$ be a natural transformation. The natural transformation induced by the recovered morphism coincides with η:*

$$(\eta^\sharp)^*_X \;=\; \eta_X \qquad \textit{for every type } X. \tag{16}$$

Proof. By lean. □

Remark 4.7 (Idea of the argument)**.** This is the substantive half of the correspondence. For an arbitrary element $p = \langle s,\, v\rangle \in \operatorname{Ext} C\, X$, naturality of η at the function $v : P_C(s) \to X$ applied to the generic element gives

$$\eta_X\langle s,\, v\rangle \;=\; \operatorname{Ext} D\, v\Big(\eta_{P_C(s)}\langle s,\, \mathrm{id}\rangle\Big), \tag{17}$$

because $\operatorname{Ext} C\, v\, \langle s, \mathrm{id}\rangle = \langle s, v\rangle$. The right-hand side is exactly $(\eta^\sharp)^*_X\langle s, v\rangle$ by definition of the induced map. Thus η is completely determined by its values on generic elements—a Yoneda-style reconstruction.

Theorem 4.8 (Representation theorem)**.** *For any containers C and D, there is a bijection*

$$\{\ \textit{container morphisms } C \Rightarrow D\ \} \;\cong\; \{\ \textit{natural transformations } \operatorname{Ext} C \Longrightarrow \operatorname{Ext} D\ \}, \tag{18}$$

implemented by $\varphi \mapsto \varphi^*$ in one direction and $\eta \mapsto \eta^\sharp$ in the other. The two constructions are mutually inverse by Theorems 4.4 and 4.6.

Proof. By lean. □

5 Discussion

What has been established. Containers are combinatorial presentations of a class of polynomial functors. The representation theorem says that this presentation is faithful at the level of morphisms: natural transformations between extensions are no more and no less than maps of shapes with position pullbacks. In particular, there are no "exotic" natural transformations that would require additional data beyond Definition 3.1.

Relation to the Yoneda lemma. The reconstruction of a natural transformation from its values on generic elements is an instance of the same idea that underlies the Yoneda lemma: a natural transformation out of a representable (or, here, polynomial) functor is determined by where it sends universal arrows. The generic element $\langle s, \mathrm{id}\rangle$ plays the role of the identity morphism at the object corresponding to shape s.

Scope. The development treats a single universe of types and does not address size issues, universe polymorphism, or the interaction of containers with dependent type formers beyond the dependent sum used in Ext. It also does not yet impose directedness, comonad structure, or other refinements; those belong to subsequent developments built on the same foundations.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant; we have not reproduced the formal proof scripts here. A mathematician reading this note need only verify that the English statements match the intended mathematics; the formal system is responsible for the derivations.

Acknowledgements

The mathematical content follows the theory of containers introduced by Abbott, Altenkirch, and Ghani. The presentation here is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.

[2] M. Abbott, T. Altenkirch, and N. Ghani. Categories of containers. In *FoSSaCS*, volume 2620 of *LNCS*, pages 23–38. Springer, 2003.